

Data Visualization

Principles and Matplotlib

Dario Pescini

Università degli Studi di Milano-Bicocca

Department of Statistics and Quantitative Methods

dario.pescini@unimib.it

Why Visualize Data?

Numbers alone can deceive. Consider four datasets with **identical** summary statistics:

	I	II	III	IV
$\bar{x}$	9.0	9.0	9.0	9.0
$\bar{y}$	7.50	7.50	7.50	7.50
$\text{Var}(x)$	11.0	11.0	11.0	11.0
r	0.816	0.816	0.816	0.816

⁰

Anscombe, F. J. (1973). Graphs in Statistical Analysis. *The American Statistician*, 27(1), 17–21.

doi:10.1080/00031305.1973.10478966

Why Visualize Data?

Numbers alone can deceive. Consider four datasets with **identical** summary statistics:

	I	II	III	IV
$\bar{x}$	9.0	9.0	9.0	9.0
$\bar{y}$	7.50	7.50	7.50	7.50
$\text{Var}(x)$	11.0	11.0	11.0	11.0
r	0.816	0.816	0.816	0.816

Anscombe's quartet

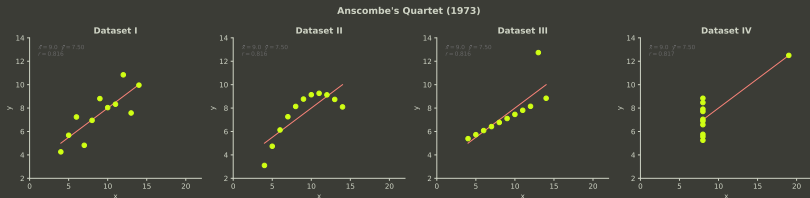
The four datasets look **completely different** when plotted. A plot reveals structure that summary statistics conceal.

⁰Anscombe, F. J. (1973). Graphs in Statistical Analysis. *The American Statistician*, 27(1), 17–21.

doi:10.1080/00031305.1973.10478966

Why Visualize Data?

Numbers alone can deceive. Consider four datasets with **identical** summary statistics:

⁰

Anscombe, F. J. (1973). Graphs in Statistical Analysis. *The American Statistician*, 27(1), 17–21.

doi:10.1080/00031305.1973.10478966

What a Good Plot Communicates

A plot is not decoration — it is a **communication tool**.

A good plot shows

- **Pattern** in the data
- **Uncertainty** and variability
- **Outliers** and anomalies
- **Relationships** between variables

A good plot avoids

- Distorted scales or proportions
- Color used as decoration
- Missing axis labels and units
- More ink than information

What a Good Plot Communicates

A plot is not decoration — it is a **communication tool**.

A good plot shows

- **Pattern** in the data
- **Uncertainty** and variability
- **Outliers** and anomalies
- **Relationships** between variables

A good plot avoids

- Distorted scales or proportions
- Color used as decoration
- Missing axis labels and units
- More ink than information

“Above all else show the data.” — Edward Tufte

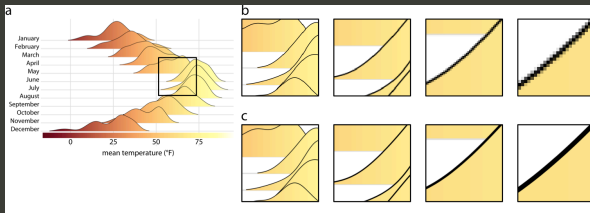
Raster vs. Vector Graphics

Raster

- Grid of pixels, each with a color
- Resolution fixed at creation
- Enlarging $\Rightarrow$ visible pixels
- Formats: PNG, JPEG, TIFF

Vector

- Geometric description (lines, curves, polygons)
- Infinitely scalable, no quality loss
- Formats: PDF, SVG, EPS



Which Format to Use?

Destination	Format	Why
Journal / thesis	PDF or SVG	Vector, editable
Presentation (Beamer)	PDF or PNG	Vector or high-res raster
Web page	SVG or PNG	Browser-native
Photo or bitmap source	PNG	Lossless raster

Rule of thumb

Use **vector** (PDF/SVG) for any plot produced from code. Use raster only when the output is inherently a bitmap (microscopy images, gel scans).

A plot is ...

A plot is a 2D mapping of properties of your data.

It is composed by distinct interacting components with their own properties:

- **Data** or the values you want to represent
- **Aesthetic** or the how to represent quantities on the "paper" (color, dimension, ...)
- **Plot type** or what I do want to represent: comparison, relations evolution in time ...

Data: type

Identify it's nature:

- **quantitative:**
 - continuous
 - discrete
- **qualitative:**
 - ordered
 - unordered
- **time** or date
- generic **text**

Data: scale

The type of the variable strongly influence [how you can map it](#).

Type of variable	Examples	Appropriate scale
quantitative continuous	1.3, 5.7, 83, $1.5 \cdot 10^{-2}$	continuous
quantitative discrete	1, 2, 3, 4	discrete
qualitative unordered	dog, cat, fish	discrete
qualitative ordered	good, fair, poor	discrete
date or time	Jan. 5 2018, 8:03am	continuous or discrete
text	The quick brown fox jumps over the lazy dog.	none or discrete

Aesthetic

We have 4 aesthetic/mapping tools at disposal:

- **position** the placement of the values in the plot
- **shape** the kind of the graphical identifier, marker or line
- **color** the color used to represent value
- **size** the dimension of the marker or the thickness of the line

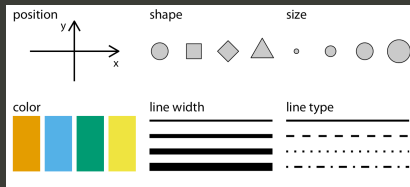
Each of these is a dimension that we can add to our 2D mapping

Aesthetic

We have 4 aesthetic/mapping tools at disposal:

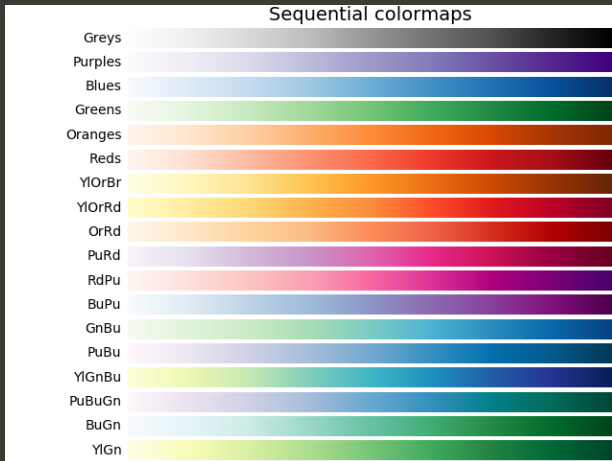
- **position** the placement of the values in the plot
- **shape** the kind of the graphical identifier, marker or line
- **color** the color used to represent value
- **size** the dimension of the marker or the thickness of the line

Each of these is a dimension that we can add to our 2D mapping



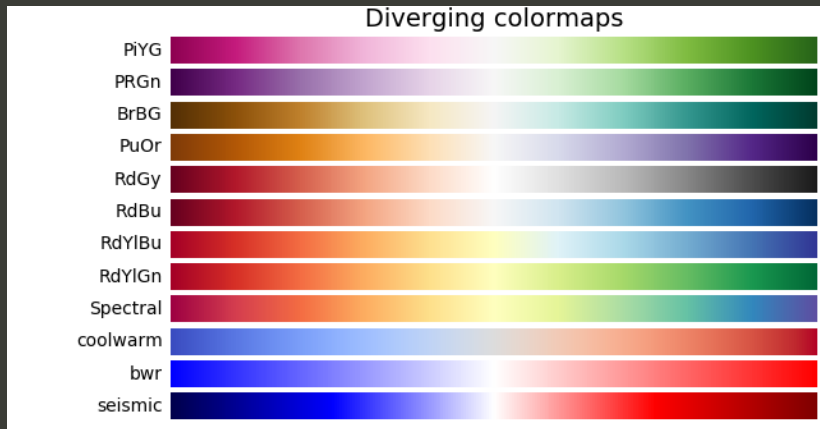
Color Scale: Sequential

Quantitative - Ordered *one direction*



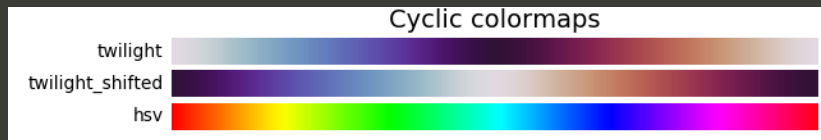
Color Scale: Diverging

Quantitative - Ordered two directions from a center



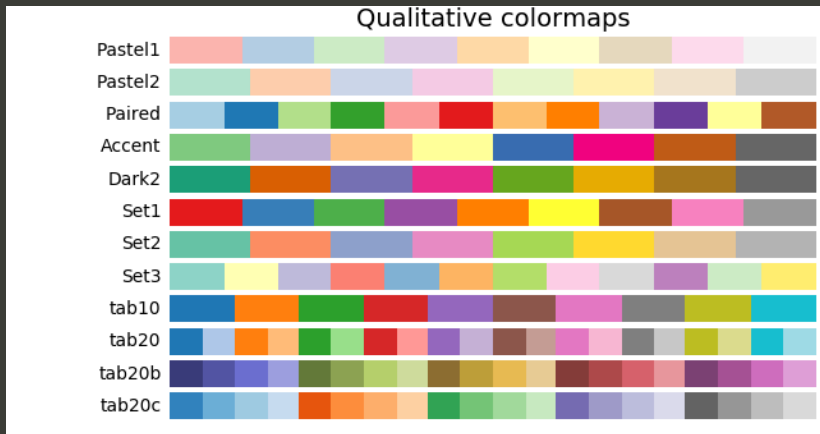
Color Scale: Cyclic

Quantitative *periodic*



Color Scale: Qualitative

Qualitative unordered



Perceptual Uniformity



Avoid the rainbow (jet)

The classic jet colormap is **not perceptually uniform**: the yellow-to-green transition appears much larger than equal data steps. It creates **false structures**.

Prefer **viridis**, **plasma**, or **ColorBrewer** palettes.

Color: Practical Rules

1 Match the scale to the data type

Sequential for magnitudes; diverging when there is a meaningful zero; qualitative for categories.

2 Use color to encode one variable

Do not use color as pure decoration. If color encodes nothing, set it to a neutral constant.

3 Fewer hues is better

Vision reliably distinguishes at most 6–8 hues. Beyond that, use facets or line styles.

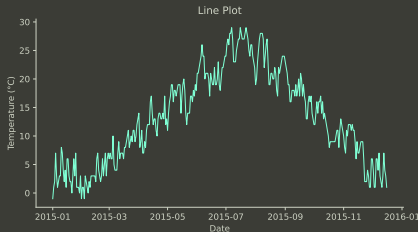
4 Think about colorblindness

~ 8% of men cannot distinguish red from green. Test with `viridis`, `cividis`, or `ColorBrewer Set2`.

Choosing the Right Plot

Question	Data	Plot
How does X change over time?	continuous × time	Line plot
Is X related to Y?	continuous × continuous	Scatter
How are groups distributed?	continuous × categorical	Box / Violin
What is the distribution of X?	continuous	Histogram
How do categories compare?	categorical × continuous	Bar chart
Uncertainty around a trend?	continuous	Error bar / Ribbon
Pairwise matrix of values?	matrix	Heatmap

Line Plot



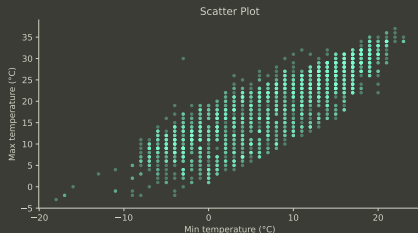
Connects points in order of x .

Ideal for **time series** and continuous functions.

Biology uses

Gene expression over time, growth curves, ODE model trajectories, survival curves

Scatter Plot



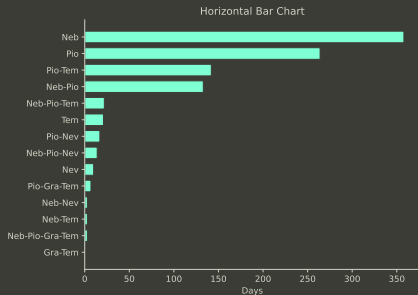
Represents pairs of values as points.

Shows **bivariate relationships**.

Biology uses

RNA-seq volcano plots, PCA scores, correlation of expression levels, sequence length vs GC content

Bar Chart

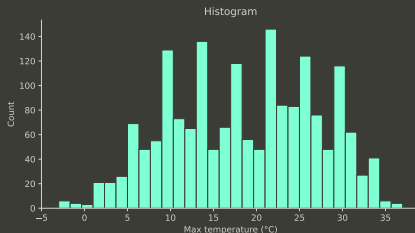


Compares values across
categorical groups.

When to use

- Grouping variable is **categorical** (months, species, conditions)
- Compare a **single aggregated value** per group
- Avoid for continuous x-axes — use a line plot instead

Histogram

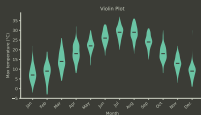
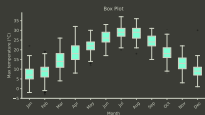


Shows the **distribution** of a single continuous variable.

Biology uses

Read length distributions, GC content, p-value histograms, expression level distributions, fragment sizes

Box Plot & Violin Plot



Compare **distributions** across groups.

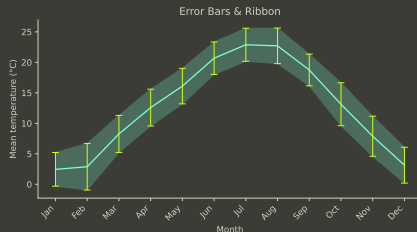
Box plot

Median, Q1, Q3, whiskers ($1.5 \times \text{IQR}$), outliers. Compact, but hides the shape.

Violin plot

Adds a kernel density estimate. Reveals bimodality and skewness invisible in a box.

Error Bars & Ribbons

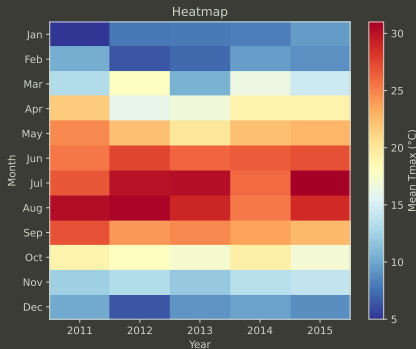


Visualize **uncertainty** around a central estimate.

Biology uses

Mean $\pm$ SD across replicates, confidence intervals on ODE trajectories, credible intervals in Bayesian parameter estimates

Heatmap



Encodes a **matrix** of values as a color grid.

Biology uses

Gene expression matrices (rows = genes, cols = samples), pairwise sequence similarities, Hi-C contact maps, correlation matrices

Matplotlib: Three Levels

Figure

The canvas. **Outermost container**; holds one or more Axes.

Axes

A single panel with its own coordinate system. This is where data is drawn by one or more artists.

(Not to be confused with *Axis* — the individual x/y ruler.)

Artists

Every visible element: lines, patches, text, ticks, spines, ...



Matplotlib: Three Levels

Figure

The canvas. **Outermost container**; holds one or more **Axes**.

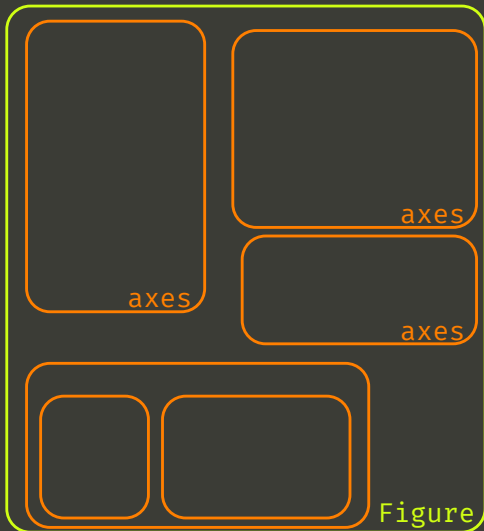
Axes

A single panel with its own coordinate system. This is where data is drawn by one or more artists.

(Not to be confused with `Axis` — the individual x/y ruler.)

Artists

Every visible element: lines, patches, text, ticks, spines, ...



Matplotlib: Three Levels

Figure

The canvas. Outermost container; holds one or more Axes.

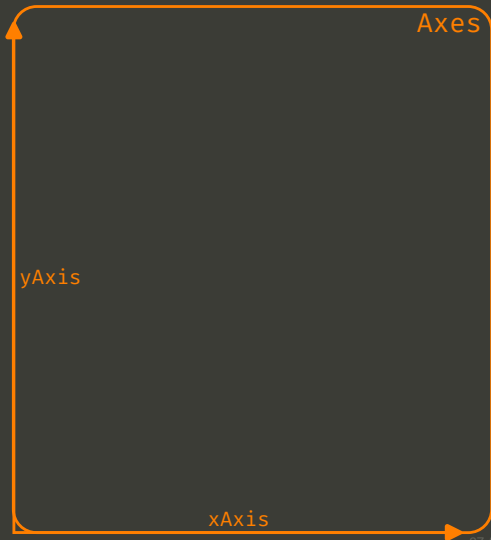
Axes

A single panel with its own coordinate system. This is where data is drawn by one or more artists.

(Not to be confused with Axis — the individual x/y ruler.)

Artists

Every visible element: lines, patches, text, ticks, spines, ...



Matplotlib: Three Levels

Figure

The canvas. Outermost container; holds one or more Axes.

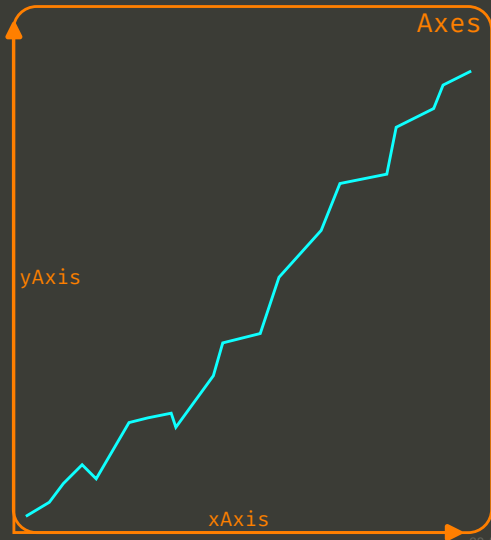
Axes

A single panel with its own coordinate system. This is where data is drawn by one or more artists.

(Not to be confused with Axis — the individual x/y ruler.)

Artists

Every visible element: lines, patches, text, ticks, spines, ...



Matplotlib: Three Levels

Figure

The canvas. Outermost container; holds one or more Axes.

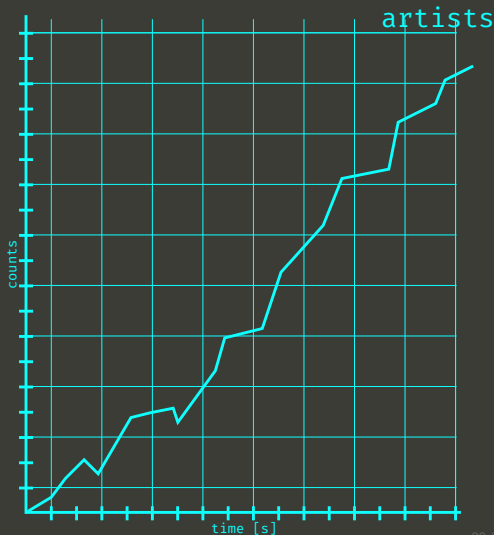
Axes

A single panel with its own coordinate system. This is where data is drawn by one or more artists.

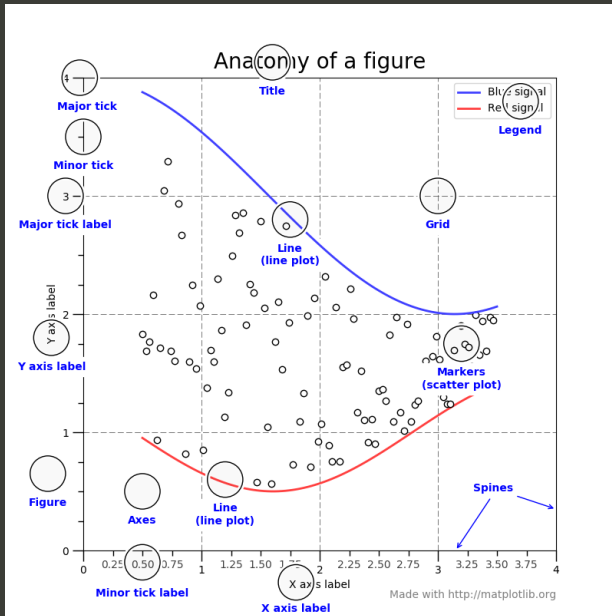
(Not to be confused with Axis — the individual x/y ruler.)

Artists

Every visible element: lines, patches, text, ticks, spines, ...



Anatomy of a Figure



The Object-Oriented API

Always prefer the **object-oriented (OO) API**:

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()      # create Figure and Axes
ax.plot(x, y)                 # draw on the axes
ax.set_xlabel("Time")
ax.set_ylabel("Temperature (°C)")
ax.set_title("Milan - daily temperature")
fig.savefig("output.pdf")
```

Why OO?

- Each ax is an explicit object — no hidden global state
- Works correctly with multiple subplots
- Matches the design of every other Python library

Multiple Panels — `subplots()`

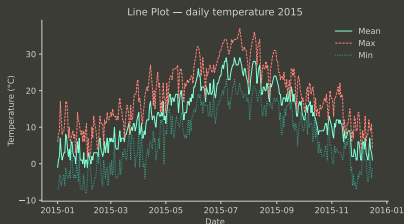
```
fig, axes = plt.subplots(nrows=2, ncols=2,
                          figsize=(10, 7))
axes[0, 0].plot(x, y1)
axes[0, 1].scatter(x, y2)
axes[1, 0].bar(categories, values)
axes[1, 1].hist(data, bins=30)

fig.tight_layout()    # avoid overlapping labels
fig.savefig("panels.pdf")
```

`axes` is a 2D NumPy array indexed by `[row, col]`.

`fig.tight_layout()` adjusts spacing automatically.

Line Plot — `ax.plot()`

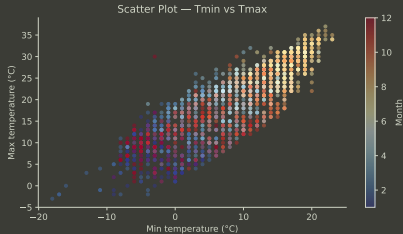


```
ax.plot(dates, tmean,  
        color="steelblue",  
        linewidth=1.2,  
        label="Mean")  
ax.plot(dates, tmax,  
        color="#E41A1C",  
        linewidth=0.8,  
        linestyle="--",  
        label="Max")  
ax.legend()
```

Key parameters

color — linewidth — linestyle ("—", "-", ":", "-.") — marker
("o", "s", ...) — label

Scatter Plot – `ax.scatter()`

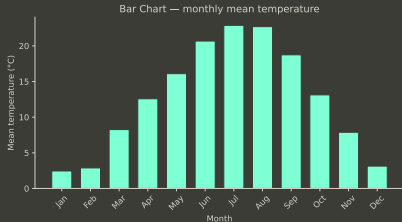


```
sc = ax.scatter(tmin, tmax,  
               c=month,  
               cmap="viridis",  
               alpha=0.6,  
               s=20)  
fig.colorbar(sc, ax=ax,  
             label="Month")
```

Key parameters

`c` – per-point color value `cmap` – colormap `s` – per-point size (pt²)
`alpha` – transparency

Bar Chart — `ax.bar()` / `ax.barh()`



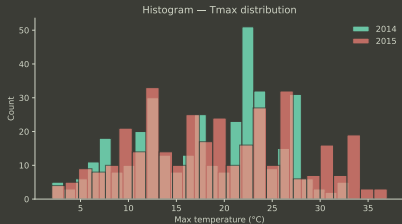
```
ax.bar(months, mean_temp,  
        color="steelblue",  
        edgecolor="white",  
        width=0.7)
```

```
# Horizontal variant  
ax.barh(months, mean_temp,  
         color="steelblue")
```

Key parameters

color — edgecolor — width (bar) / height (barh) — align
("center", "edge")

Histogram — `ax.hist()`



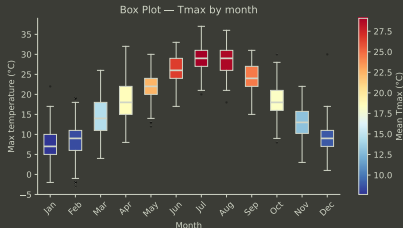
```
ax.hist(tmax, bins=30,  
        color="steelblue",  
        edgecolor="white",  
        density=False)
```

```
# Overlapping distributions  
ax.hist(tmax_2014, bins=25,  
        alpha=0.6,  
        label="2014")  
ax.hist(tmax_2015, bins=25,  
        alpha=0.6,  
        label="2015")  
ax.legend()
```

Key parameters

`bins` — number of bars `density` — normalize to area = 1 `alpha` — overlap distributions `edgecolor`

Box Plot — `ax.boxplot()`

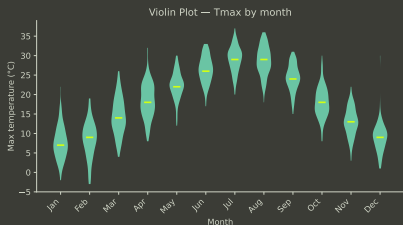


```
ax.boxplot(data_by_month,  
  
            tick_labels=month_names,  
            patch_artist=True,  
            medianprops=dict(  
                color="white",  
                linewidth=2))
```

Key parameters

`patch_artist` — fill boxes with color `medianprops`, `whiskerprops`,
`boxprops` — style dicts `tick_labels`

Violin Plot — `ax.violinplot()`

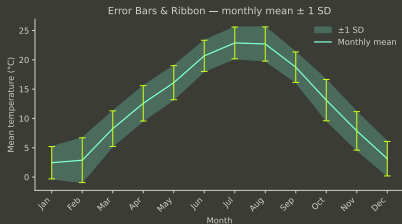


```
vp = ax.violinplot(  
    data_by_month,  
    positions=range(1,  
        13),  
    showmedians=True,  
    showextrema=False)  
for body in vp["bodies"]:  
  
    body.set_facecolor("steelblue")  
    body.set_alpha(0.7)
```

Key parameters

`showmedians` — draw median line `showextrema` — draw min/max lines
`vp["bodies"]` — list of `PolyCollection` patches

Error Bars & Ribbons

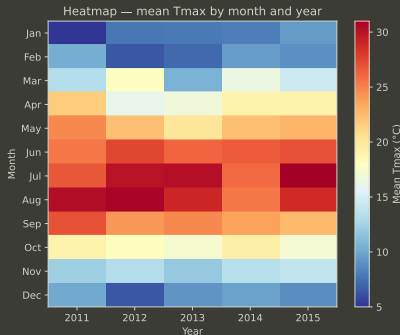


```
ax.errorbar(x, mean,  
            yerr=std,  
            fmt="o",  
            capsize=4,  
            color="steelblue")  
  
ax.fill_between(dates,  
                mean - std,  
                mean + std,  
                alpha=0.25)  
ax.plot(dates, mean)
```

Key parameters

yerr — error magnitude fmt — marker+line style capsize — errorbar cap width alpha — ribbon transparency

Heatmap — `ax.imshow()`



```
im = ax.imshow(matrix,  
                aspect="auto",  
                cmap="RdBu_r",  
                vmin=-3,  
                vmax=3)  
fig.colorbar(im, ax=ax,  
             label="log$_2$  
             FC")  
ax.set_xticklabels(  
    sample_names,  
    rotation=45, ha="right")
```

Key parameters

`cmap` — colormap `vmin/vmax` — color range `aspect` — cell shape
("auto", "equal") `fig.colorbar()` — color scale bar

Saving Figures

```
fig.savefig("temperature.pdf",  
            bbox_inches="tight")    # no clipping  
  
fig.savefig("temperature.svg",  
            bbox_inches="tight")  
  
fig.savefig("temperature.png",  
            dpi=300,  
            bbox_inches="tight")
```

Format	Use case	dpi needed?
.pdf	journals, Beamer slides	no
.svg	web, editable vector	no
.png	presentations, web images	yes (≥ 300)
.eps	older journal systems	no

Publication-Ready Checklist

Always include

- Axis labels with **units**
- Legend (if multiple series)
- Title or caption
- Font size ≥ 10 pt for labels

Avoid

- Gratuitous grid lines
- 3D effects and shadows
- Truncated y-axis (usually)
- Top and right spines (add visual noise)

```
ax.set_xlabel("Date",      fontsize=12)
ax.set_ylabel("Temp (°C)",  fontsize=12)
ax.tick_params(labelsize=10)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
```

Further Reading

- **Fundamentals of Data Visualization** — Claus O. Wilke

<https://clauswilke.com/dataviz/>

- **Ten Simple Rules for Better Figures**

Rougier et al., *PLOS Comput. Biol.* 2014

- **Matplotlib documentation & cheatsheets**

<https://matplotlib.org/cheatsheets/>

- **ColorBrewer 2.0** — interactive palette chooser

<https://colorbrewer2.org>